AI 文 旅 路 线 规 划 微 信 小 程 序

漫途
技术文档

从一篇攻略，到一条能跟着走的线

本文档描述「漫途」小程序的项目定位、需求分析、系统架构、AI 技术方案、 核心功能实现、测试部署与创新点，用于参赛评审与技术交接。

项目类型	技术栈	文档版本	日期
微信小程序 · AI 应用	原生小程序 / Node.js / Ollama	V1.0	2026 年 10 月

目录

CONTENTS

一、项目概述 3

1.1 项目背景　1.2 项目定位　1.3 建设目标　1.4 交付形态

二、需求分析 5

2.1 目标用户　2.2 核心痛点　2.3 功能需求　2.4 非功能需求　2.5 需求边界

三、系统架构设计 8

3.1 总体架构　3.2 页面结构　3.3 数据流　3.4 存储设计　3.5 接口设计

四、AI 技术方案 12

4.1 模型选型　4.2 四类能力　4.3 提示词策略　4.4 输出结构化　4.5 兼容与降级　4.6 成本与合规

五、核心功能实现 16

5.1 发现与分流　5.2 攻略与讲解　5.3 一键行程　5.4 路线定制　5.5 发布与社区　5.6 账号与资产

六、测试与部署 21

6.1 测试环境　6.2 功能测试　6.3 接口测试　6.4 降级测试　6.5 静态检查　6.6 部署

七、创新点说明 25

八、总结与展望 27

阅读说明：本文档中的界面图为产品实际运行界面；数据规模以当前版本为准（22 个页面、5 个页签、17 条攻略、17 个目的地）。 所有技术描述均对应项目源码实现，未做能力外推。

适用范围：参赛评审、技术评审、后续开发交接。

一 项目概述

1.1 项目背景

文旅内容在社交平台上并不稀缺，稀缺的是**可执行的行程**。用户在小红书、马蜂窝看到一篇写得不错的攻略， 通常要经历三步损耗：先把长文读完，再把散落的时间、地点、花费在脑子里拼成一条路线，最后自己打开地图重新核一遍交通。 三步走完，热情往往已经消耗了一半。

另一方面，大模型已经能承担“把结构化信息重组成叙述”这件事，但大多数旅行类 AI 产品把它做成了聊天框： 用户需要自己想问题、自己描述偏好，输出的又常是一段无法直接使用的文字。 **问题不在于模型不够强，而在于 AI 没有被放进具体的产品动作里。**

1.2 项目定位

漫途是一款 AI 文旅路线规划微信小程序。它的定位可以用一句话概括： **把一篇攻略，变成一条能跟着走的线。**

围绕这句话，产品把 AI 拆成四个具体动作，而不是一个通用对话入口： **讲一篇攻略**（AI 讲解）、**排一条路线**（AI 路线定制）、 **优化一次发布**（攻略 AI 建议）、**引一段他人经验**（博主参考卡片）。

1.3 建设目标

- **降低阅读成本**：把长文攻略转成可切换主题的口语化讲解，用户按“怎么走 / 吃什么 / 住哪里 / 避坑”提问，而不是自己翻找。
- **降低规划成本**：用六个选择题替代自由文本输入，30 秒内产出带时间、交通、分项预算的按天行程。
- **缩短从看到到出发的链路**：攻略详情页提供“按这篇走”主按钮，一键把内容派生成行程，而不是止步于收藏。
- **保证演示与离线可用**：AI 服务或本地后台不可用时，产品核心功能仍然完整可用，不出现白屏或功能缺失。

1.4 交付形态与技术选型

表 1-1　交付形态与选型

交付物	形态	选型理由
用户端	微信小程序（原生 wxml / wxss / js / json）	免安装、分享链路短；原生开发便于精细控制交互与体积，不引入第三方组件库
AI 推理	本地 Ollama + deepseek-r1:7b	中文理解与推理能力足够；本地部署不产生调用费用，赛事演示环境可控
服务层	Node.js 原生 http 服务（零第三方依赖）	统一 AI 返回格式、提供攻略增删改查；单文件即可启动，部署成本接近零
数据层	小程序本地缓存 + 服务端 JSON 文件	无需数据库与云环境，离线可用；演示时不依赖网络

二 需求分析

2.1 目标用户与使用场景

表 2-1　目标用户画像

用户类型	典型特征	核心诉求	对应能力
决策困难型	有假期、没想法，只知道想出去	别让我从零开始想，给我几条现成的线	首页本月路线、AI 路线定制、按主题分类
效率优先型	已经选定目的地，需要落地成计划	告诉我几点去哪、怎么去、大概花多少	攻略详情、AI 讲解、按这篇走、分项预算
经验分享型	刚玩回来，有表达欲	填一次表就能发出去，还希望有人帮我润色	发布攻略、AI 优化建议、社区广场

2.2 核心痛点

1. **攻略读完用不上**：长文以叙述为主，时间、交通、花费散落在段落里，读者需要二次加工。
2. **规划门槛高**：自由文本式的 AI 对话要求用户“会提问”，表达不清就得到泛泛而谈的结果。
3. **内容与行动断裂**：主流产品的终点是收藏夹，收藏之后没有任何下一步。
4. **AI 结果不可信**：用户无法分辨哪句话来自真实经验、哪句话是模型编的。
5. **演示环境脆弱**：依赖云端服务的 AI 功能，一旦断网或额度用尽，整个演示链路断裂。

2.3 功能需求

表 2-2　功能需求清单（P0 为必须，P1 为重要）

编号	需求	说明	优先级	页面
FR-01	首页发现	本月推荐路线轮播、搜索、主题八宫格、三个能力入口	P0	index
FR-02	攻略浏览	按旅行方式分类（城市漫步 / 自然风光 / 美食 / 海岛），共 17 条	P0	guides
FR-03	攻略详情	路线 / 正文 / 花费 / 贴士四分区，作者、季节、预算、节奏摘要	P0	guide-detail
FR-04	AI 讲解	按五个主题生成口语化讲解，逐字呈现，支持重听与切换主题	P0	ai-guide
FR-05	一键行程	把攻略派生成按天行程，含交通、分项预算、节奏	P0	itinerary
FR-06	AI 路线定制	六项结构化输入，输出按天路线 + 预算 + 交通 + 避坑 + 博主参考	P0	ai-plan
FR-07	发布攻略	12 项编号表单，提交后入社区并异步请求 AI 优化建议	P0	publish
FR-08	社区广场	卡片流展示价格、路程、交通、月份、人群标签，支持进详情	P0	community
FR-09	个人资产	收藏、浏览足迹、行程草案、草稿的统一入口与统计	P1	mine / favorites / history
FR-10	账号与资料	微信头像昵称填写登录，资料本地持久化，重开不重置	P1	login / profile
FR-11	内容管理	后台连接状态、攻略删除、演示数据重置	P1	admin

2.4 非功能需求

表 2-3　非功能需求与实现约束

维度	要求	落地方式
响应性	AI 请求最长可达数十秒，界面不得假死	请求超时放宽至 120 s；生成中显示“正在生成”状态；逐字上屏制造进度感
可用性	AI 或本地后台不可用时，产品核心流程仍可完成	所有写操作先落本地缓存，再尝试同步；失败静默降级并提示
兼容性	适配 iOS / Android 常见机型与微信版本	仅使用小程序基础能力（wx.request / Storage / FileSystem）；不依赖组件库
可维护性	页面之间不重复实现同一套逻辑	演示数据与工具函数抽取到 `utils/`，多页面复用
数据留存	用户资料、收藏、足迹在重启后不丢失	写入 `wx.setStorageSync`，应用启动时回填全局状态
合规	AI 生成内容需可辨识，票价等信息需标注以官方为准	AI 建议独立分区展示；博主卡片署名并注明来源

2.5 需求边界

本期**不包含**：在线支付与预订、真实用户体系的账号打通（登录为演示级）、多人协同编辑行程、 语音合成播报（讲解为文字呈现）、UGC 图片上传与审核。上述内容列入第八章演进路线。

三 系统架构设计

3.1 总体架构

系统采用四层结构，自上而下为表现层、能力层、服务层与模型层，另有一个贯穿全程的本地数据层。 小程序端不直接访问模型，所有 AI 请求经本地 Node 服务统一转发与格式化，**端侧只承诺一种返回结构。**

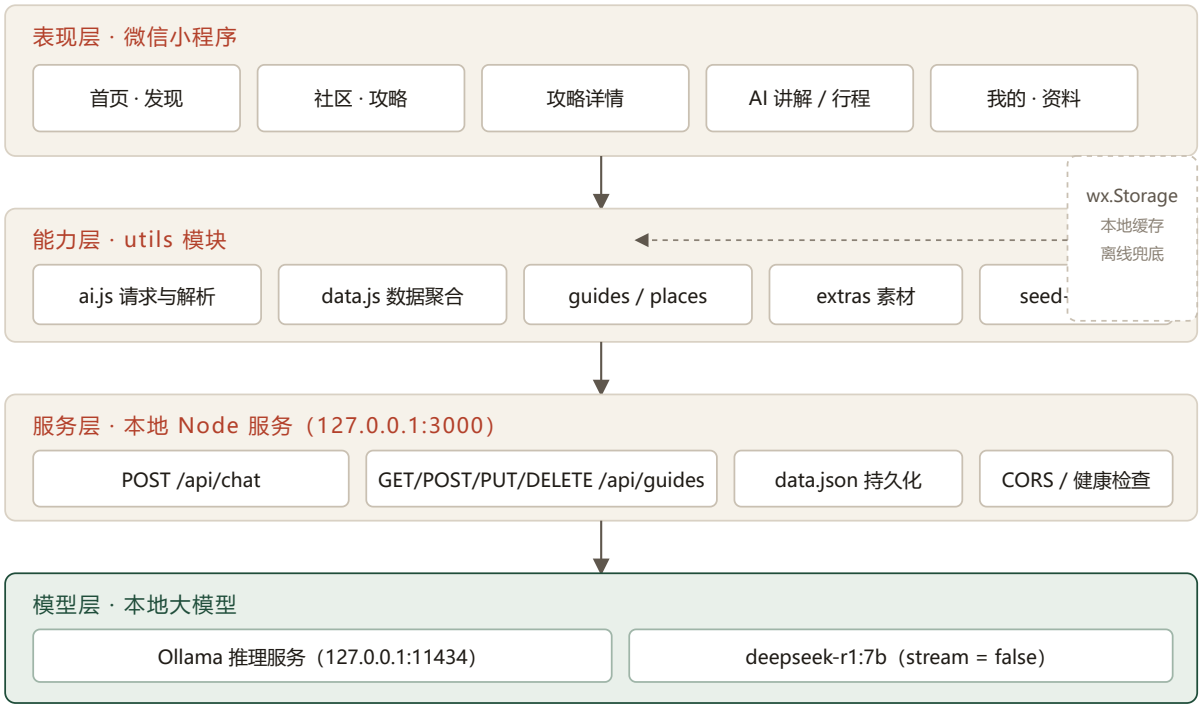


图 3-1　系统总体架构：四层结构 + 本地缓存兜底

3.2 页面结构与模块划分

小程序共注册 22 个页面，其中 5 个为页签常驻页面。页面按职责划分为内容、AI 能力、用户资产、管理四组， 避免一个页面承担过多职责。

表 3-1　页面分组（共 22 页）

分组	页面	职责
页签常驻	`index / community / guides / destinations / mine`	一级入口，承载分发
内容消费	`guide-detail / destination-detail / community-detail / search`	详情与检索
AI 能力	`ai-guide / ai-plan / itinerary / guide-create / publish`	讲解、定制、行程、创作
用户资产	`login / profile / favorites / history / settings`	身份与留存数据
支撑页面	`admin / about / guide-list`	管理、说明、草稿列表

3.3 关键数据流

以“AI 讲解”为例，一次完整请求跨越四层，端侧只认一种返回结构，因此模型或桥接层更换不影响前端：



图 3-2　AI 讲解请求链路与失败降级路径

3.4 数据存储设计

本期不使用云数据库。小程序端以本地缓存为主存储，服务端以 JSON 文件持久化，二者通过同步接口保持一致； 任何一侧不可用都不会阻塞主流程。

表 3-2　本地存储键设计

键名	内容	写入时机	说明
`user_profile`	昵称、签名、城市、头像、登录态	登录 / 保存资料	启动时回填全局状态，重开不重置
`favorite_ids`	收藏的攻略 id 数组	点击收藏	与内存状态双向同步
`view_history`	浏览记录（id + 时间戳）	打开详情页	按 id 去重置顶，上限 50 条
`my_guides`	用户自建攻略	创建 / 编辑	攻略草稿来源
`community_guides`	社区攻略列表	发布 / 后台同步	空时回填内置演示数据

3.5 服务接口设计

表 3-3　本地服务接口

方法	路径	入参	返回
`POST`	`/api/chat`	`{ model, message }`	`{ reply }`，异常时附带 `error`
`GET`	`/api/guides?type=my|community`	—	`{ list }`
`POST`	`/api/guides?type=…`	攻略对象	`{ ok, id }`，同 id 去重后置顶
`PUT`	`/api/guides?type=…`	`{ list }`	`{ ok, total }`，整体替换（重置用）
`DELETE`	`/api/guides?type=…&id=…`	—	`{ ok }`
`GET`	`/api/health`	—	`{ ok, name, time }`

四 AI 技术方案

4.1 模型选型与部署形态

模型选用 **deepseek-r1:7b**，通过本地 **Ollama** 部署（默认端口 11434）。 选型基于三点：一是中文语料理解与长文本重组能力满足“把攻略讲成话”的需求；二是 7B 参数量在消费级设备上可运行， 演示现场不依赖公网与调用额度；三是推理过程可控，便于在端侧过滤其思维链输出。

推理以非流式方式进行（`stream: false`），一次返回完整文本。这是刻意的取舍： **简化前端解析，换取稳定。**生成中的体感由前端逐字动画补足，而不是依赖流式协议。

4.2 四类 AI 能力的输入输出设计

表 4-1　AI 能力矩阵

能力	输入	输出形态	落点
攻略讲解	攻略标题、正文、花费、贴士 + 主题标签	口语化段落，按主题切换	ai-guide 页
路线定制	目的地、天数、预算区间、偏好、同行、节奏	按天行程 + 交通 + 分项预算 + 避坑	ai-plan → itinerary
攻略优化	发布表单的关键词与行程描述	润色与补充建议，回写 `aiTips`	社区攻略详情
博主参考	目的地 + 主题	署名卡片，附来源说明	路线定制结果页

4.3 提示词与上下文组织策略

- **角色前置**：每次请求首句固定角色设定（如“你是一位文旅路线规划助手”），收敛输出语气与视角。
- **事实打底**：把攻略的结构化字段（标题、目的地、亮点关键词、行程、避雷）按固定顺序拼进 Prompt，让模型的输出有可依据的原文，而不是自由发挥。
- **粒度约束**：讲解类请求只针对**一个主题**，不要求一次讲完所有内容； 拆分后每次输出更短、更聚焦，也更符合“被带着看”的体验。
- **任务化指令**：明确要求“润色、补充建议、优化路线安排、补充避坑提示、输出精简中文”， 避免模型输出大段无关背景介绍。

4.4 输出结构化与前端呈现

大模型的自然语言输出被约束进固定容器，是这套方案能落地的关键。以路线定制为例，模型结果被拆进四个模块：

表 4-2　路线定制结果的模块化拆分

模块	内容	量纲 / 约束
按天行程	逐日的时段、地点、动作说明	天数为输入上限，不越界
交通与耗时	出发地到首站的方式、分钟数、费用	分钟 + 元
分项预算	住宿 / 餐饮 / 交通 / 门票四项区间	须落在用户输入的预算区间内
出发前提示	预约规则、时段、体力等避坑条目	3 条以内，短句

讲解类能力则采用**主题切片**：总览 / 怎么走 / 吃什么 / 住哪里 / 避坑五个标签， 每次只请求一个标签，输出直接替换正文区域。用户看到的是“换了个话题继续讲”，而不是一整段重新生成。

4.5 兼容、容错与降级

- **响应多格式兼容**：解析层依次尝试 `reply / response / answer / text / message / message.content` 及嵌套 `data`， 任一命中即采用，避免上层封装变化导致整条链路失效。
- **思维链过滤**：移除模型输出中的 `<think>…</think>` 段落，只保留正式回答，避免推理过程泄漏到界面。
- **超时分级**：端侧请求 120 秒，服务端转发 180 秒，端侧先于服务端超时，保证用户总能收到明确反馈而不是无限等待。
- **失败不阻塞主流程**：发布、收藏等写操作先落本地，再异步同步；AI 建议失败时，攻略照常存在，详情页提供自动补拉与手动重试。
- **服务不可达提示**：明确提示检查本地服务与开发者工具的域名校验开关，而不是抛出一句笼统的“网络错误”。

> 设计原则
>
> AI 是增强项，不是前置条件。**任何一次模型调用失败，都不能让用户丢失已经产生的数据。** 这条原则同时决定了数据流顺序：先落库，后调模型。

4.6 成本、隐私与合规

本地部署使调用成本为零，且用户输入的行程内容不出本机，隐私风险低。合规上采取两项措施： AI 生成内容在界面上独立分区，并标注为 AI 建议；涉及票价、开放时间等信息时， 以**“以官方为准”**的说明收尾，避免把模型输出当作权威事实。

五 核心功能实现

5.1 发现与分流

首页承担“替用户做第一步判断”的职责。顶部以时间维度给出本月适合出发的路线，主推一条大图路线卡， 下方用八个单字入口（城、野、岛、味、南、远、文、东）做主题分类，再往下是三张能力卡：AI 路线定制、攻略社区、我的攻略。

实现上，首页数据由 `utils/guides.js` 与 `utils/places.js` 聚合而成（当前各 17 条）， 轮播与卡片在同一数据源上做视图裁剪，避免多处维护同一份内容。搜索入口跳转独立检索页， 支持按目的地、攻略标题与季节关键词匹配。



图 5-1 首页：本月路线、主题入口、能力卡片

5.2 攻略详情与 AI 讲解

详情页把一篇攻略拆成**路线、正文、花费、贴士**四个分区，顶部固定作者卡与四项摘要 （最佳季节、人均预算、适合人群、节奏）。底部常驻三个动作：收藏、分享、按这篇走， 其中“按这篇走”是唯一的实心主按钮。

进入 AI 讲解后，页面顶部提供五个主题标签。用户点击任一标签，前端组装该攻略的正文摘要与该主题，发起一次模型请求；返回文本经思维链过滤后，由定时器逐字写入数据层，形成“正在讲解”的观感。 结束后提供“再说一遍”，并提示可切换主题听其他讲法。

技术上需要注意的是：一次请求的结果会被缓存于页面数据中，同一主题重复切换不重复请求，降低等待与推理开销。



图 5-2　详情页：四分区 + AI 讲解入口

5.3 一键生成行程

行程页是内容到行动的转换节点。页面顶部给出天数、预算、节奏三项摘要与一条紧凑的路线条 （伏见稻荷 → 锦市场 → 祇园 → 鸭川），中部按天展开安排，底部把总预算拆成交通、住宿、餐饮、门票四个条形项。

实现上，行程由攻略数据派生：天数取自攻略，路线取自分区中的地点序列，预算按既定比例拆分到四类。 **派生物与源内容保持双向可达**——页面底部保留“回到原攻略”，行程不是攻略的替代品。



图 5-3　行程页：天数、路线条、逐日安排、分项预算

5.4 AI 路线定制

表单只包含六组输入：目的地、天数（滑块）、预算区间（下限—上限）、偏好（多选标签）、同行、节奏。 **六项全部是选择，没有自由文本**，这一决策直接降低了用户表达成本，也让模型输入始终处于可控范

围。

“预算区间”而非单一数字，是一个关键的小改动：模型获得的是范围约束， 生成结果自然落在合理区间内，也便于结果页输出分项区间（如住宿 ¥240–480）。

结果页按“一天一天 → 预算怎么分 → 大交通怎么到 → 出发前看一眼 → 博主怎么说”的顺序组织， 底部提供重填、换一版、存到我的行程三个动作。**“换一版”是容错设计**： 同组参数可重新生成，用户不必从头填表。



图 5-4　定制表单：六项输入全部为选择

5.5 攻略发布与社区广场

发布表单包含 12 个字段，按“结构化信息在前、故事在后”排序：标题、目的地、路程时长、预算、门票、出行方式、 适合人群、出行月份、亮点关键词、避雷提醒、详细行程、昵称。字段顺序与社区卡片的信息顺序一一对应， 填完即可生成一张完整卡片。

提交流程遵循“先落库、后调模型”：先将攻略写入本地缓存并同步本地后台，社区列表立即置顶刷新； 随后异步请求模型生成优化建议，回写至该条攻略的 `aiTips` 字段，在详情页独立分区展示。 模型未启动时发布仍然成功，仅提示后台未连接。

社区卡片集中展示五个比较维度：人均价格、路程时长、交通方式、出发月份、适合人群， 并附话题标签与作者昵称。列表按发布时间倒序，新发布内容即时置顶。



图 5-5　发布页：12 项编号字段



图 5-6　社区广场：价格、路程、交通、人群标签

5.6 账号体系与用户资产

账号体系使用微信原生能力：头像选择（`open-type="chooseAvatar"`）与昵称填写（`type="nickname"`）， 登录成功后将资料写入 `user_profile`，应用启动时回填全局状态，**重新打开小程序资料不重置。** 头像通过文件系统接口转存为本地持久文件，避免临时路径失效。

“我的”页把用户行为沉淀为三类资产：**收藏**（想去）、**足迹**（去过）、**草案**（正在计划）。 足迹为真实记录：每次打开攻略详情写入一条 `{id, ts}` 记录，按 id 去重置顶，上限 50 条， 列表页每次进入重新读取，按相对时间展示。

内容管理页提供后台连接状态、攻略删除与演示数据重置，作为运营侧的最小可用后台； 后台在线时以后台数据为准并回写本地，离线时直接操作本地缓存。

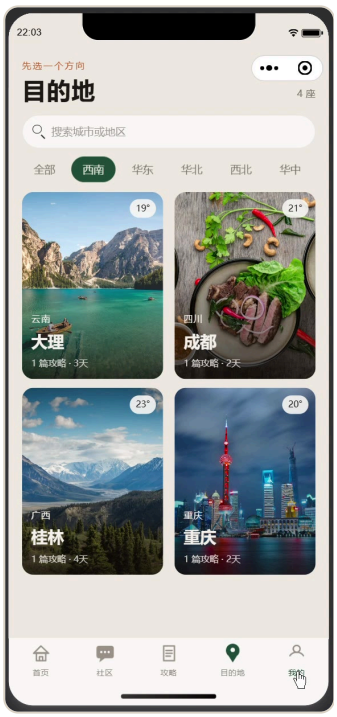


图 5-7　我的：收藏、足迹、草案三类资产

六 测试与部署

6.1 测试环境

表 6-1　测试环境

项目	配置
运行环境	微信开发者工具模拟器（iOS / Android 双端机型）、Windows 11
服务端	Node.js 本地服务，监听 127.0.0.1:3000
模型服务	Ollama，监听 127.0.0.1:11434，模型 deepseek-r1:7b
网络设置	开发者工具「不校验合法域名」开启（本地 HTTP 接口必需）

6.2 功能测试

表 6-2　功能测试用例与结果

编号	用例	步骤	预期	结果
TC-01	首页加载与分类切换	进入首页 → 点击主题入口	各分类列表正确渲染	通过
TC-02	攻略详情分区切换	进入详情 → 依次切换四个分区	内容与分区对应，无错位	通过
TC-03	AI 讲解主题切换	点击五个主题标签	对应讲解内容替换，可重听	通过
TC-04	一键生成行程	详情页点击“按这篇走”	生成按天行程与分项预算	通过
TC-05	路线定制完整流程	填写六项 → 生成 → 换一版 → 保存	结果结构完整，二次生成不重填	通过
TC-06	攻略发布与刷新	填写 12 项 → 发布 → 返回广场	新攻略置顶，卡片信息完整	通过
TC-07	资料持久化	修改资料 → 重启小程序	资料与登录态保留	通过
TC-08	浏览足迹记录	浏览多篇攻略 → 打开足迹页	倒序展示，重复浏览置顶不重复	通过
TC-09	收藏持久化	收藏 → 重启 → 查看收藏页	收藏项保留	通过
TC-10	内容管理	删除单条 / 重置演示数据	列表与广场同步生效	通过

6.3 接口测试

表 6-3　服务接口冒烟测试

方法	用例	断言	结果
GET	/api/health	返回 `ok = true` 与服务标识	通过
POST	/api/guides 新增一条	返回 `ok` 与新记录 id	通过
GET	/api/guides 读取列表	新记录位于首位	通过
DELETE	/api/guides 删除该记录	返回 `ok`，总数回落	通过
POST	/api/chat 正常推理	返回统一 `{reply}` 结构	通过

6.4 降级与异常测试

表 6-4　异常场景与预期行为

异常场景	系统行为	结果
本地后台未启动	写操作成功落本地缓存，提示“后台未连接，已存本地”	通过
模型服务未启动	发布照常完成，AI 建议为空并保留重试入口	通过
请求超时	到达超时后给出明确失败提示，界面不假死	通过
端口被占用	服务启动时提示端口冲突，引导关闭旧进程	通过
空数据状态	社区与足迹为空时回填演示数据，页面不留白	通过

6.5 静态检查

- 全部 JavaScript 文件通过语法检查（20 余个文件），JSON 配置可正常解析。
- 全项目源码扫描无 UTF-8 BOM 标记，避免 WXSS / WXML 编译异常。
- 页面注册与实际目录一一对应（22 个页面，无遗漏、无冗余）。

6.6 部署步骤

表 6-5　运行环境部署

步骤	操作	说明
1	安装 Node.js	任意 18+ 版本，用于运行本地服务
2	安装并启动 Ollama	拉取 `deepseek-r1:7b`，确认 11434 端口可访问
3	启动本地服务	在服务目录执行启动脚本，控制台输出接口地址
4	导入小程序项目	微信开发者工具导入项目根目录，填写 AppID
5	关闭域名校验	详情 → 本地设置 → 勾选“不校验合法域名”
6	验证	访问 `/api/health`，在「我的 → 攻略管理后台」查看连接状态

> 已知限制
>
> 未进行大规模真机兼容测试；服务端数据为单文件存储，不具备并发写入能力； 登录为演示级实现（未接入服务端换取 openid）； AI 输出质量受本地设备算力影响。

七 创新点说明

7.1 内容形态：可切换主题的攻略讲解

常规做法是把攻略压缩成一段 AI 摘要，用户读完仍要自己找重点。本项目的做法是 **把讲解拆成五个主题**——总览、怎么走、吃什么、住哪里、避坑，一次只讲一个。 用户不是在读摘要，而是在按自己的疑问逐条追问，**同一篇攻略可以被讲出五种版本。**

这一形态的价值在于：它保留了原文的结构与事实，同时把“检索式阅读”变成了“对话式收听”， 阅读成本显著下降，且不需要用户自己组织问题。

7.2 交互范式：结构化输入替代自由 Prompt

多数 AI 旅行产品要求用户写一段需求描述，结果质量高度依赖表达能力。本项目把输入固化为 **六个选择题**（目的地、天数、预算区间、偏好、同行、节奏）， 用户无需会写提示词，模型输入也始终处于可控范围。

配合“换一版”，形成**“低成本重试”**的闭环：不满意只需再点一次， 而不是重新填表。这是把生成式交互做进产品、而非做成聊天框的关键一步。

7.3 可信度分层：AI 说的与人说的分开放

结果页把两类信息明确分开： **“出发前看一眼”**是模型建议， **“博主怎么说”**是署名卡片并注明来源，页面底部统一给出“以官方为准”的说明。

生成式产品最常见的信任问题是“用户不知道哪句能信”。分层的意义不是削弱 AI， 而是让用户知道每类信息该信到什么程度——这比把所有内容混在一起更有说服力。

7.4 工程可靠性：AI 可缺席的产品设计

项目把“AI 不可用”当作常态而非异常来设计：写操作先落本地再同步， 模型失败不影响发布，详情页保留自动补拉与手动重试，服务未启动时给出可操作的排查指引。

结果是：拔掉模型，产品仍然是一个完整的攻略与社区应用；接上模型，才多出四类 AI 能力。 这种“能力可插拔”的架构，使演示环境与真实弱网场景都不至于翻车。

7.5 端云协同：轻量服务层 + 本地兜底

自建零依赖 Node 服务承担两件事：统一 AI 返回结构、提供攻略的增删改查。 端侧只承诺一种协议，模型或桥接层更换不影响任何页面代码；同时本地缓存作为兜底， 即使服务层完全不在，功能也不缺失。

这套结构在赛事场景中额外收益明显：部署成本接近零，单文件即可启动，评审现场无需依赖公网与额度。

八 总结与展望

8.1 完成情况

当前版本完成了从内容发现到行程生成、再到 UGC 发布的完整闭环， 共交付 22 个页面、5 个页签、17 条攻略与 17 个目的地数据，四类 AI 能力全部接入并可运行。 工程上建立了统一的 AI 请求协议、本地缓存兜底机制与内容管理后台。

表 8-1　目标达成情况

目标	达成情况	状态
降低阅读成本	五主题 AI 讲解，可切换、可重听	已达成
降低规划成本	六项选择生成按天行程，支持换一版	已达成
缩短决策链路	详情页“按这篇走”一键派生行程	已达成
保证离线可用	服务与模型缺席时功能完整，写操作不丢	已达成
内容可信度	AI 建议与署名来源分区展示	已达成

8.2 不足

- **数据规模有限**：攻略与目的地各 17 条，长尾目的地覆盖不足。
- **行程为静态派生**：尚未与真实交通时刻、天气、开放时间联动，缺少冲突检测。
- **AI 输出未做结构化校验**：预算与耗时由模型生成，缺少规则层兜底与数值合理性检查。
- **账号体系为演示级**：未接入服务端会话，多设备不同步。
- **内容审核缺失**：UGC 无自动审核与举报链路。

8.3 演进路线

表 8-2　三阶段规划

阶段	主题	关键任务
近期	结果可信化	为 AI 输出增加规则校验（预算上下限、耗时合理性）；接入天气与景区开放时间；补充冲突检测
中期	账号与数据打通	服务端会话与 openid，收藏、足迹、草案跨设备同步；内容审核与举报机制
远期	多模态与服务化	行程语音播报、拍照识景点、行中实时调整；开放攻略数据接口，支持第三方分发

结语

漫途想解决的不是“AI 会不会写攻略”，而是**“读完一篇攻略之后，人为什么还是没出发”**。 把 AI 拆进发现、阅读、规划、发布四个真实动作里，让每一步都少一点阻力——这是这个项目全部设计的出发点。